# Research note: LLD practice with explainable feedback

Public-source research, finalized 15 October 2026. The linked product pages and documentation were reviewed again on this date. Paid features were not tested, and no learner interviews or learning-outcome study were conducted.

## Learner problem

The target learner understands basic object-oriented programming and wants to practise an interview-sized design. They can propose classes for a Parking Lot or Vending Machine, but still need to judge whether responsibilities are cohesive, interactions satisfy requirements, and the design can accommodate change. Several designs may be defensible, so resemblance to a reference answer is an incomplete basis for feedback.

A meaningful attempt should retain assumptions, responsibilities, interactions, failure cases, and trade-offs. The learner needs to understand which decisions to keep, which to reconsider, and what to try next. Preserving previous answers and feedback lets revision become part of the practice experience.

## Existing approaches and implications

**Hello Interview: guided LLD practice.** Its public catalogue describes step-by-step practice with personalized feedback and shows Start and History actions for some problems. This supports problem selection, guidance, feedback, and revisiting attempts as an established practice loop. Our prototype applies that loop at a smaller scope; the catalogue does not establish the quality or internal workings of its assessments. 1

**Educative: Grokking LLD.** The course overview describes OOP/SOLID foundations and design problems developed through requirements, object relationships, UML diagrams, implementation, and mock interviews. This suggests that learners benefit from a repeatable way to express their reasoning. Our five-section submission captures a subset of that evidence in text. The public overview alone cannot establish every capability of the paid course. 2

**Exercism: iterative feedback and mentoring.** Its documentation distinguishes immediate automated feedback on submitted iterations from targeted volunteer mentoring and discussion, which can involve a wait. Although Exercism concerns coding practice more broadly, it illustrates complementary automated and human feedback approaches. Our product separates deterministic validity checks from design judgement and retains each attempt. 3

Exercism's mentoring guidance recommends prioritizing one to three observations per iteration and giving enough explanation to help learners progress without supplying the whole solution. This informs our three-improvement limit and the emphasis on actionable suggestions. 4

## Opportunity and evidence limits

The opportunity is a focused experience where a learner can trace a judgement to their own submitted reasoning and use a short improvement list to guide revision. These sources support the interaction principles, but do not prove that competitors lack evidence-based feedback or that our approach improves learning. The product direction is a hypothesis to validate through use.

## Product direction and chosen scope

The implemented MVP offers Parking Lot, Vending Machine, and Library Management, one demo learner, and five required text sections: requirements/assumptions, classes/responsibilities, relationships/interactions, edge cases/testing, and trade-offs. Text keeps the prototype manageable while allowing supporting pseudocode. It does not establish that code would execute correctly or capture every relationship as clearly as a diagram.

Deterministic rules check completeness, size limits, and valid submission states. A rubric-guided LLM considers requirements coverage, responsibilities, relationships/interfaces, extensibility/trade-offs, and testability. Feedback uses Strong, Needs improvement, or Insufficient evidence, with an explanation, suggestion, and up to three priorities. Strong and Needs improvement must cite literal text from the learner's answer. The prompt asks the evaluator to consider justified alternatives fairly.

For example, if an answer assigns allocation, pricing, and payment to one class, useful feedback would cite that decision, explain a resulting change or testing difficulty, and suggest reconsidering the responsibility boundary. This is an illustrative coaching example, not a finding about a researched product.

The learner can save a draft, submit, see evaluation status, review feedback, and create a new prefilled attempt. Earlier answers and feedback remain accessible in History. Failed evaluation preserves the submitted answer and offers a manual retry. Login, executable code, diagram editing, and side-by-side comparison are outside the MVP.

## What has been checked and what remains

Automated checks cover the practice lifecycle, persistence, validation, and failure/retry behaviour. Browser checks use clearly labelled simulated feedback. One live Parking Lot evaluation produced five assessments that passed the rubric validator and were saved successfully; it also received a manual relevance review. This demonstrates one working example and does not establish consistent judgement, fairness across alternative designs, or resistance to embedded instructions.

The next validation should compare two defensible solutions and an intentionally weak one, then observe a learner completing a feedback/revision cycle. Ask whether they can explain each priority, act on it, and recognize what improved. Such work would test the product hypothesis; no learning gain is claimed here.

## Sources


1. Hello Interview: Low-Level Design Guided Practice.
2. Educative: Grokking the Low Level Design Interview Using OOD Principles.
3. Exercism: Getting Feedback.
4. Exercism: How to give great feedback.